

Ampliación de Bases de Datos

Grado en Ingeniería del Software

Curso académico 2026/2027

TPI 3

Acceso a datos en Aula Viva: JDBC y arquitectura de tres niveles

Estudiante: Miguel Garcia Galan
Profesor: Dr. Anier Soria Lorente
Fecha: 09/11/2026
Repositorio: https://github.com/miguelitillo14/tpi3-aula-viva

Universidad Loyola Andalucía

Índice

1. Problema real y contexto **2**

2. Necesidad relacionada con los datos y objetivos **2**

2.1. Necesidad identificada 2

2.2. Objetivo general 2

2.3. Objetivos específicos 2

3. Conceptos y tecnologías aplicadas **3**

3.1. Arquitectura de tres niveles 3

3.2. Independencia de datos 3

3.3. JDBC, ODBC y ORM 3

4. Desarrollo del trabajo **3**

4.1. Diseño de la solución 4

4.2. Implementación 4

5. Resultados obtenidos **5**

6. Análisis e interpretación **5**

7. Decisiones tomadas y justificación **6**

8. Conclusiones y aprendizaje **6**

9. Declaración de uso de inteligencia artificial **7**

Referencias **7**

A. Reproducción del prototipo **7**

1. Problema real y contexto

Aula Viva es una propuesta de aplicación para que los estudiantes consulten espacios de estudio del campus y sus informes recientes de ocupación y ruido. Para utilizarla, la aplicación debe leer información persistente del catálogo de espacios y de las observaciones registradas. El TPI 2 abordó la integración analítica de esas fuentes con Spark SQL; este trabajo estudia el acceso de una aplicación a la base relacional que sirve la información de consulta.

El problema concreto es ofrecer a la aplicación una interfaz estable y de solo lectura para consultar el estado reciente de un espacio, sin exponer directamente la estructura interna de la base ni conceder permisos de escritura a la cuenta de consulta. La demostración emplea PostgreSQL y datos sintéticos. No representa una conexión al sistema real del campus ni utiliza datos personales.

2. Necesidad relacionada con los datos y objetivos

2.1. Necesidad identificada

La aplicación necesita localizar espacios y recuperar su estado más reciente de forma segura y comprensible. Para ello hay que separar la interfaz que consume el usuario de la representación relacional y del almacenamiento físico, y definir un mecanismo de conexión que no mezcle datos del usuario con el texto SQL.

2.2. Objetivo general

Diseñar y demostrar un prototipo de acceso de solo lectura a PostgreSQL para Aula Viva, explicando los niveles externo, conceptual e interno, y aplicando JDBC desde una aplicación Java.

2.3. Objetivos específicos

- Modelar espacios e informes mediante claves, restricciones y una relación uno a muchos.
- Publicar una vista externa que entregue a la aplicación solo los campos necesarios.
- Consultar esa vista con JDBC y parámetros preparados.
- Distinguir independencia lógica y física mediante cambios posibles que preservan el contrato de consulta.

- Obtener y analizar resultados reproducibles con datos de demostración.

3. Conceptos y tecnologías aplicadas

3.1. Arquitectura de tres niveles

El nivel externo es la vista `app.vw_student_space_status`: presenta el nombre del espacio, capacidad, ocupación, ruido, hora del informe y plazas estimadas disponibles. Es el contrato que consume la aplicación. El nivel conceptual lo forman las entidades `space` y `report`, sus atributos, claves, relación y restricciones. El nivel interno incluye los tipos físicos de PostgreSQL, sus páginas y los índices B-tree definidos para las consultas.

3.2. Independencia de datos

La independencia lógica permite cambiar el esquema conceptual sin obligar a cambiar al consumidor, siempre que se mantenga la vista externa acordada. Por ejemplo, se podrían separar atributos de edificio en otra tabla y conservar la vista mediante una unión. La independencia física permite cambiar índices o parámetros de almacenamiento sin alterar tablas lógicas ni la salida de la vista. En la demostración se crea un índice B-tree compuesto para la búsqueda del último informe; la aplicación sigue ejecutando la misma consulta.

3.3. JDBC, ODBC y ORM

La implementación usa JDBC, la API estándar de Java para abrir una conexión, preparar SQL y leer filas. Los valores del filtro de espacio se envían como parámetros de `PreparedStatement`, no se concatenan en la consulta. ODBC es una interfaz alternativa, especialmente útil para herramientas o lenguajes distintos de Java; no se implementa aquí porque la aplicación de ejemplo está escrita en Java.

Hibernate es un ORM que puede mapear filas y relaciones a objetos de dominio y reducir operaciones repetitivas de persistencia. En este prototipo se elige JDBC directo para que se vean con claridad la consulta, los permisos y la vista externa. Hibernate sería una alternativa para una aplicación con más entidades y lógica de persistencia; no forma parte de la ejecución presentada.

4. Desarrollo del trabajo

4.1. Diseño de la solución

El flujo de lectura es: aplicación Java → JDBC → vista externa de PostgreSQL → tablas relacionales. La aplicación se conecta como rol `aula_reader`, que solo tiene permiso de lectura sobre la vista. El usuario administrador crea el esquema, las tablas, las restricciones, los datos sintéticos y el índice. La cuenta de lectura no recibe permisos sobre las tablas base.

Nivel	Elemento en Aula Viva	Responsabilidad
Externo	`vw_student_space_status`	Contrato de consulta limitado para la aplicación
Conceptual	`space`, `report`, clave foránea y restricciones	Representar espacios y observaciones
Interno	Tipos PostgreSQL, páginas e índices B-tree	Organizar y acelerar el acceso físico

4.2. Implementación

El script `db/init/001_schema.sql` crea dos tablas: cada espacio puede tener varios informes, y cada informe referencia un espacio existente. Las restricciones limitan la ocupación a $[0, 1]$, el ruido a un rango definido para la muestra y la capacidad a valores positivos. La vista calcula el informe más reciente por espacio. El índice B-tree compuesto sobre `(space_id, reported_at DESC, report_id DESC)` apoya el patrón temporal de consulta.

La clase Java lee los parámetros de conexión de variables de entorno, abre JDBC con `DriverManager`, usa `PreparedStatement` y cierra los recursos con `try-with-resources`. Un argumento opcional permite filtrar por identificador de espacio. Docker Compose prepara PostgreSQL y ejecuta el cliente. Las credenciales declaradas en `docker-compose.yml` son valores públicos de demostración para el entorno local; no son secretos reales y no deben reutilizarse en producción.

```
String sql = """
    SELECT space_id, building, space_name, capacity,
           occupancy_ratio, noise_db, reported_at,
              available_places
    FROM app.vw_student_space_status
    WHERE space_id = ?
    """;
try (Connection connection = DriverManager.getConnection(url,
   user, password)) {
    connection.setReadOnly(true);
```

```
    try (PreparedStatement statement = connection.
       prepareStatement(sql)) {
        statement.setString(1, spaceId);
        try (ResultSet result = statement.executeQuery()) {
            while (result.next()) {
                System.out.println(result.getString("space_id")
                    + " | " + result.getString("space_name")
                    + " | " + result.getBigDecimal("
                       occupancy_ratio"));
            }
        }
    }
}
```

Listing 1: Consulta parametrizada de la vista externa

5. Resultados obtenidos

La carga inicial contiene cuatro espacios y siete informes sintéticos; A101, A102 y B201 tienen más de un informe. La vista devuelve un solo estado vigente por espacio. La ejecución comprobada del cliente sin filtro devolvió cuatro filas; al consultar `A101`, devolvió el informe `R002`, con ocupación 0,80, ruido de 50 dB y 6 plazas estimadas disponibles. La consulta de un identificador inexistente devuelve cero filas sin error.

Espacio	Informe	Edificio	Capacidad	Ocupación	Libres
A101	R002	A	30	0,80	6
A102	R007	A	20	0,25	15
B201	R006	B	15	0,50	7
C101	R005	C	40	0,90	4

La salida real incluyó los cuatro estados de la tabla y terminó con código 0. La consulta filtrada de `A101` devolvió una fila con R002. Estos resultados permiten comprobar la selección del último informe, la relación entre tablas y el consumo desde JDBC de una vista. No son mediciones reales del campus ni una evaluación de carga o rendimiento a escala.

6. Análisis e interpretación

La vista ofrece una forma de acceso más estable que consultar directamente tablas: puede conservar sus columnas aunque cambie la organización interna, siempre que se

mantenga el contrato. El rol de lectura limita el daño potencial de una consulta mal planteada. El uso de parámetros evita construir SQL con texto aportado por el usuario. El índice se justifica por el acceso que busca el último informe de un espacio, pero su beneficio no se ha medido en esta muestra pequeña.

El prototipo no implementa autenticación de estudiantes, una interfaz web, JDBC con el sistema institucional real, ODBC ni Hibernate. Tampoco prueba una migración de esquema ni una carga concurrente. Esas funciones requieren decisiones y sistemas fuera del alcance de esta demostración; la defensa debe distinguir el comportamiento ejecutado de las extensiones propuestas.

7. Decisiones tomadas y justificación

- **PostgreSQL:** permite definir relaciones, restricciones, vistas e índices en una base relacional reproducible localmente.
- **JDBC:** es el mecanismo directo de acceso desde Java y permite mostrar explícitamente conexión, consulta parametrizada y lectura del resultado.
- **Vista externa de solo lectura:** desacopla al consumidor de tablas internas y restringe los campos expuestos.
- **Datos sintéticos:** permiten repetir la prueba sin acceder a información privada del campus.
- **Hibernate y ODBC como alternativas:** se explican para comparar sus papeles, pero no se añaden dependencias que no son necesarias en este cliente pequeño.

8. Conclusiones y aprendizaje

El prototipo conecta una aplicación Java con PostgreSQL mediante JDBC y consulta un contrato externo de solo lectura. La separación entre vista, tablas relacionales y almacenamiento físico permite explicar los tres niveles y razonar sobre la independencia lógica y física. Las claves y restricciones protegen la relación entre espacios e informes; la consulta preparada filtra de forma segura y la vista entrega el último estado conocido.

La prueba demuestra el acceso funcional con datos sintéticos. No demuestra una integración con la infraestructura real del campus ni ventajas de rendimiento del índice. Para evolucionar el sistema habría que validar el contrato con el responsable de los datos, desplegar credenciales gestionadas y medir consultas con una carga representativa.

9. Declaración de uso de inteligencia artificial

Se utilizó ChatGPT como apoyo para organizar y revisar la memoria, y para preparar el prototipo y los datos sintéticos. El código y los resultados descritos se comprobaron mediante la ejecución local del proyecto. El autor es responsable del contenido final.

Referencias

[1] Soria Lorente, A., *Trabajos Prácticos Individuales: Orientaciones para el alumnado, Ampliación de Bases de Datos, curso 2026/2027*, Universidad Loyola Andalucía, apartado 7: TPI 3, Tema 6.

[2] PostgreSQL Global Development Group, *PostgreSQL 16 Documentation*. https://www.postgresql.org/docs/16/

[3] Oracle, *JDBC Database Access: Prepared Statements*. https://docs.oracle.com/javase/tutorial/jdbc/basics/prepared.html

[4] Hibernate, *Hibernate ORM 6.6 User Guide*. https://docs.hibernate.org/orm/6.6/userguide/html_single/

[5] Garcia Galan, M., *Aula Viva: propuesta inicial del proyecto*, Universidad Loyola Andalucía, 2026.

A. Reproducción del prototipo

Desde PowerShell, situado en la carpeta del proyecto, ejecutar:

```
docker compose up --build --abort-on-container-exit
docker compose run --rm app A101
docker compose down
```

Listing 2: Arranque y consulta desde PowerShell

El primer arranque descarga las imágenes y compila el cliente Java dentro del contenedor. El primer comando muestra los cuatro estados y detiene los servicios al terminar el cliente. El segundo inicia la base de datos si es necesario y muestra solo A101. El último cierra los servicios y conserva el volumen de demostración.